Лабораторная работа 6. Решение уравнений и систем уравнений

Подготовил студент группы 523 Чех Степан

```
import numpy as np
import scipy as sp
```

Задание 1

Задайте две матрицы 3-го порядка с помощью функции rand. Объедините их в новую матрицу двумя способами: сначала по горизонтали, а потом по вертикали.

```
arr3rd_1 = np.random.rand(3,3)
arr3rd_2 = np.random.rand(3,3)
display(arr3rd_1)
display(arr3rd_2)
display(np.hstack((arr3rd_1, arr3rd_2))) # Сложение по горизонтали
display(np.vstack((arr3rd_1, arr3rd_2))) # Сложение по вертикали
```

```
array([[0.10869185, 0.4264938 , 0.0713964 ],
       [0.30144465, 0.31939123, 0.61311053],
       [0.2572936 , 0.38107985, 0.80576704]])
array([[0.89320456, 0.10064432, 0.75181002],
       [0.7449746 , 0.18476436, 0.38756476],
       [0.34233529, 0.4255129 , 0.9569885 ]])
array([[0.10869185, 0.4264938 , 0.0713964 , 0.89320456, 0.10064432,
        0.75181002],
       [0.30144465, 0.31939123, 0.61311053, 0.7449746 , 0.18476436,
        0.38756476],
       [0.2572936 , 0.38107985, 0.80576704, 0.34233529, 0.4255129 ,
        0.9569885 ]])
array([[0.10869185, 0.4264938 , 0.0713964 ],
       [0.30144465, 0.31939123, 0.61311053],
       [0.2572936 , 0.38107985, 0.80576704],
       [0.89320456, 0.10064432, 0.75181002],
       [0.7449746 , 0.18476436, 0.38756476],
       [0.34233529, 0.4255129 , 0.9569885 ]])
```

Задание 2

Задайте квадратную матрицу 5-го порядка с помощью функции rand. Удалите две первые строки в этой матрице, а затем последний столбец.

```
arr_5rd = np.random.rand(5,5)
display(arr_5rd)
arr_5rd = np.delete(arr_5rd, 0, 0)
arr_5rd = np.delete(arr_5rd, 0, 0)
arr_5rd = np.delete(arr_5rd, -1, 1)
display(arr_5rd)
```

```
array([[0.81465357, 0.90250387, 0.17983784, 0.92440378, 0.10893318],
       [0.93973953, 0.09210279, 0.95458033, 0.58853173, 0.33806189],
       [0.94389307, 0.45271151, 0.98993325, 0.42229484, 0.94920933],
       [0.48750754, 0.67801595, 0.20924449, 0.29003637, 0.77235805],
       [0.57279162, 0.65253881, 0.44455897, 0.36000904, 0.88155505]])
array([[0.94389307, 0.45271151, 0.98993325, 0.42229484],
       [0.48750754, 0.67801595, 0.20924449, 0.29003637],
       [0.57279162, 0.65253881, 0.44455897, 0.36000904]])
```

Задание 3

Выделите из квадратной матрицы 3-го порядка главную диагональ, побочную диагональ, (+1)-ю диагональ – на 1-н уровень выше главной, (-1)-ю диагональ – на 1-н уровень ниже главной.

```
arr3rd = np.random.rand(3,3)
display(arr3rd)
display(np.diag(arr3rd))
display(np.diag(np.fliplr(arr3rd)))
display(np.diag(arr3rd, k=1))
display(np.diag(arr3rd, k=-1))
```

```
array([[0.83673341, 0.20777978, 0.77308078],
       [0.4303527 , 0.92772175, 0.83452665],
       [0.54137632, 0.53813797, 0.36011185]])
array([0.83673341, 0.92772175, 0.36011185])
array([0.77308078, 0.92772175, 0.54137632])
array([0.20777978, 0.83452665])
array([0.4303527 , 0.53813797])
```

Задание 4

Создайте матрицу 8-го порядка из 1 и 0 в шахматном порядке.

```
arr8rd = np.zeros((8,8))
for i in range(8):
    for j in range(8):
        # Если сумма индексов чётная, записываем 1
        if (i+j)%2 == 0:
            arr8rd[i][j] = 1
        else:
            arr8rd[i][j] = 0
display(arr8rd)
```

```
array([[1., 0., 1., 0., 1., 0., 1., 0.],
       [0., 1., 0., 1., 0., 1., 0., 1.],
       [1., 0., 1., 0., 1., 0., 1., 0.],
       [0., 1., 0., 1., 0., 1., 0., 1.],
       [1., 0., 1., 0., 1., 0., 1., 0.],
       [0., 1., 0., 1., 0., 1., 0., 1.],
       [1., 0., 1., 0., 1., 0., 1., 0.],
       [0., 1., 0., 1., 0., 1., 0., 1.]])
```

Задание 5

Выполните преобразования матрицы, используя встроенные функции и/или операторы цикла while или for (варианты см. ниже). Вариант 6. Создайте матрицу 3-го порядка, заполненную целыми случайными числами. Измените матрицу следующим образом: а) Преобразуйте матрицу в вектор-столбец. Обнулите все элементы, которые имеют нечетные значения. б) Обнулите все элементы, которые имеют четные значения. Преобразуйте матрицу в вектор-строку.

```
arr3rd = np.random.randint(0, 100, (3, 3))
display(arr3rd)
matrix_a = np.array([ arr3rd[i][j] if arr3rd[i][j] % 2 == 0 else 0 for i in range(3) for j in range(3)])
display(matrix_a.transpose())
matrix_b = np.array([ arr3rd[i][j] if arr3rd[i][j] % 2 == 1 else 0 for i in range(3) for j in range(3)])
display(matrix_b)
```

```
array([[88, 25, 25],
       [99, 15, 78],
       [10, 13,  7]], dtype=int32)
array([88,  0,  0,  0,  0, 78, 10,  0,  0])
array([ 0, 25, 25, 99, 15,  0,  0, 13,  7])
```

Задание 6

Выполните решение СЛАУ с помощью np.linalg.solve(a, b) (согласно варианту). Выполните проверку решения.

$$\begin{cases} 45x_1 - 3,5x_2 + 7,4x_3 = 25 \\ 3,1x_1 - 6x_2 - 2,3x_3 = -15 \\ 0,8x_1 + 7,4x_2 - 5x_3 = 64 \end{cases}$$

```
a = np.array([[45, 3.5, 7.4],
              [3.1, 6, 2.3],
              [0.8, 7.4, 5]])
b = np.array([25, -15, 64])
# Решаем систему уравнений Ax = b методом численного решения (LU-разложение)
x = np.linalg.solve(a,b)
display(x)
# Проверка решения: подставляем x обратно в систему
# np.dot(a, x) вычисляет матричное произведение A·x
# np.allclose() сравнивает результат с вектором b с учётом погрешности вычислений
check = np.allclose(np.dot(a, x), b) # Проверка
check
```

```
array([ -3.81909128, -13.09947713,  32.79828075])
True
```

Задание 7

Выполните решение СЛАУ матричным методом. Выполните проверку решения.

```
# Вычисляем обратную матрицу A⁻¹ и умножаем её на вектор правых частей b
x = np.dot(np.linalg.inv(a), b)
display(x)
# np.allclose() сравнивает результат с исходным вектором b, допуская машинную погрешность
check = np.allclose(np.dot(a, x), b) # Проверка
check
```

```
array([ -3.81909128, -13.09947713,  32.79828075])
True
```

Задание 8

Результаты одинаковые, оба метода сработали без проблем.

Задание 9

Решите нелинейное уравнение g(x) = 0 (по вариантам).

$x^4 + 5x^3 + 7x^2 + 7x - 20$

```
# Данный список соответствует уравнению: 1·x⁴ + 5·x³ + 7·x² + 7·x − 20 = 0
coeffs = [1, 5, 7, 7, -20]

# np.roots находит все корни многочлена (действительные и комплексные)
all_roots = np.roots(coeffs)

print("Все корни уравнения:")
for root in all_roots:
    print(f"x = {root:.6f}")  # Комплексные корни (если есть)
```

```
Все корни уравнения:
x = -4.000000+0.000000j
x = -1.000000+2.000000j
x = -1.000000-2.000000j
x = 1.000000+0.000000j
```

Задание 10

Решите систему нелинейных уравнений с помощью функций root() и fsolve().

$$\begin{cases} \sqrt{x}\cos y = 5xy + 3 \\ \dfrac{5y}{x^2 + y^2 + 1} + x - y = 4 \end{cases}$$

```
def equations(vars):
    x, y = vars # Распаковка входного вектора переменных
    f1 = np.sqrt(x) * np.cos(y) - 5*x*y - 3
    f2 = 5*y / (x**2 + y**2 + 1) + x - y - 4
    return [f1, f2]

# Начальное приближение
initial_guess = [1, 1]

# Решение через scipy.optimize.root методом Левенберга-Марквардта ('lm')
# Алгоритм минимизирует сумму квадратов невязок: 0.5 * Σ(f_i)².
sol_root = sp.optimize.root(equations, initial_guess, method='lm')

# Вывод результата с точностью до 6 знаков после запятой
print("Решение (root):")
print(f"x = {sol_root.x[0]:.6f}, y = {sol_root.x[1]:.6f}")

# Решение через scipy.optimize.fsolve
# Использует модифицированный метод Ньютона-Рафсона с численным якобианом.
sol_fsolve = sp.optimize.fsolve(equations, initial_guess)
```

```
print("\nРешение (fsolve):")
print(f"x = {sol_fsolve[0]:.6f}, y = {sol_fsolve[1]:.6f}")
```

```
Решение (root):
x = 3.964229, y = -0.051034

Решение (fsolve):
x = 3.964229, y = -0.051034
```

Вывод:

В ходе выполнения лабораторной работы были успешно выполнены все поставленные задания, связанные с матричными операциями, решением систем линейных и нелинейных уравнений, а также использованием библиотек NumPy и SciPy

Контрольные вопросы

1. Создание матриц и векторов в Python с помощью Numpy.
2. Индексация векторов и матриц в системе Matlab, удаление, обнуление строк, столбцов.
3. Поэлементные и матричные операции в Numpy.
4. Получение транспонированных массивов.
5. Объединение матриц.
6. Использование оператора цикла для создания матриц.
7. Решите СЛАУ (состоящую из 3-х уравнений) методом Гаусса / Крамера.
8. Что значит решить систему уравнений?
9. Суть метода Гаусса для решения систем линейных уравнений.
10. Суть метода Крамера для решения систем линейных уравнений.

Ответы

1. Векторы и матрицы формируются через np.array() или конструкторы вроде np.zeros(), np.ones() и np.linspace(). Библиотека автоматически преобразует вложенные списки в многомерные массивы с единым типом данных для оптимальной работы.
2. В Matlab используется индексация с единицы, а доступ к элементам задаётся круглыми скобками и двоеточием. Строки или столбцы удаляются присваиванием пустого массива [], а обнуление выполняется записью нулей в нужный срез.
3. Поэлементные действия выполняются стандартными знаками (+, *, **) и применяются к каждому элементу независимо. Для истинного матричного умножения используется оператор @ или функция np.dot(), учитывающая правила линейной алгебры.
4. Транспонирование выполняется через свойство .T или функцию np.transpose(), которые меняют местами строки и столбцы. При этом создаётся новое представление данных без физического копирования, что экономит память и время.
5. Массивы склеиваются функциями np.vstack() для конкатенации по вертикали и np.hstack() для соединения по горизонтали. Также применяется универсальная np.concatenate() с явным указанием оси объединения.
6. Циклами можно последовательно заполнять заранее выделенный массив np.empty(), вычисляя каждый элемент по заданной формуле. Хотя такой подход медленнее векторизованных аналогов, он удобен при сложных зависимых вычислениях.
7. В Python систему удобно решать функцией np.linalg.solve(A, b), которая автоматически подбирает устойчивый численный алгоритм. При ручной реализации метод Гаусса приводит матрицу к треугольному виду, а метод Крамера находит отношения определителей.
8. Решить систему означает найти такой набор значений переменных, при котором все уравнения одновременно обращаются в верные равенства. Система может иметь единственное решение, быть несовместной или обладать бесконечным множеством корней.
9. Метод Гаусса последовательно исключает неизвестные, приводя расширенную матрицу к ступенчатому виду с помощью элементарных преобразований строк. Обратный ход позволяет найти значения переменных, начиная с последнего уравнения и подставляя результаты в предыдущие.
10. Метод Крамера применим только к квадратным системам с ненулевым определителем и выражает каждую неизвестную через отношение двух определителей. Числитель получается заменой соответствующего столбца матрицы коэффициентов на вектор свободных членов.